DAY 04 · OBJECTS

Classes, Objects & Exceptions

Classes, dunder methods, exceptions, and the object-model traps.

2–3 hours / day04_classes_objects / Objects and errors

INSIDE THIS BOOKLET

Classes · Dunders · Exceptions · Mutability

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.



Read & type
Type every example. Predict the output first.



Practice
Implement or debug one test at a time.



Test
Run the suite. Read the first failing line.



Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day04_classes_objects/` · the course runner `python run.py 4`

In this session

Why classes, and why they matter for the exam 3
1 Anatomy of a class 3
2 The two big sharing traps 5
3 Properties: attributes that run code 7
4 `classmethod` and `staticmethod` 8
5 Special ("dunder") methods 9
6 Inheritance 10
7 Exceptions: signalling that something went wrong 11
8 Generators: producing values one at a time 13
9 Copying objects 14
10 The toolbox: techniques the exercises rely on 14
Exercises 17
End-of-day checklist 18

> **◎ Today's goal**
>
> The broken algorithm in the exam will very likely be a class (a tokenizer, a cache, a parser, a store...). You must read and fix classes quickly, and know the object-model traps.

Why classes, and why they matter for the exam

So far, our data and our functions have lived separately: a list here, a function that processes it there. That's fine for small tasks. But many programs need to *remember things between calls*. A cache needs to remember what it has stored. A tokenizer needs to remember the vocabulary it has learned. A key-value store needs to remember which transactions are open. Passing all of that state around as separate variables quickly becomes unmanageable. A **class** solves this by bundling some data together with the functions that work on it. You can think of a class as a *blueprint* or a *cookie cutter*: it describes what every object of that kind has and can do. An **object** (also called an **instance**) is one actual thing made from that blueprint, with its own copy of the data. The exam's broken program is very likely to be a class, because the algorithms that make good exams (tokenizers, caches, stores, parsers) all have state. Most bugs in class-based code come from one question: **which object does this piece of data belong to?** When data that should belong to one object is accidentally shared between several, you get strange, hard-to-explain failures. Today is about learning to see that.

1 Anatomy of a class

Let's build up a small class step by step, then look at the whole thing.

1.1 A first class

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
        return self.value
```

```
>>> c = Counter()        # create an object from the blueprint
>>> c.increment()
1
>>> c.increment()
2
>>> d = Counter()        # a second, independent object
>>> d.increment()
1
>>> c.value              # c kept its own value
2
```

Here's what's going on:

- `class Counter:` defines the blueprint. Nothing is created yet.
- `c = Counter()` creates a new object. Python makes an empty object and then automatically calls the special method `__init__` on it, whose job is to set up the starting state. Here, it gives the object a `value` of 0.

- `c.increment()` calls a **method**, a function that belongs to the class. Calling it on `c` makes it work on `c`'s data.
- `c` and `d` are separate objects, each with its own `value`.

1.2 What `self` is

Every method takes `self` as its first parameter. `self` is **the object the method was called on**. When you write `c.increment()`, Python actually runs `Counter.increment(c)`, so inside the method `self` *is* `c`. That's how the method knows which object's data to change. `self.value` means "the `value` belonging to this particular object". Data stored like this is called an **instance attribute**: each instance has its own.

1.3 A fuller example

Now the full example. Read it slowly, because every later section builds on these pieces.

```
class Account:
    bank_name = "PyBank"          # CLASS attribute: shared by all instances

    def __init__(self, owner, balance=0):
        self.owner = owner        # INSTANCE attributes: one per object
        self.balance = balance
        self.history = []         # create mutable state HERE, per instance

    def deposit(self, amount):    # instance method: first param is self
        if amount <= 0:
            raise ValueError("amount must be positive")
        self.balance += amount
        self.history.append(("deposit", amount))
        return self.balance

    def __repr__(self):
        return f"Account({self.owner!r}, {self.balance})"
```

```
>>> acc = Account("ana", 100)
>>> acc.deposit(50)
150
>>> acc.history
[('deposit', 50)]
>>> acc
Account('ana', 150)
>>> acc.bank_name
'PyBank'
```

Line by line:

- **`bank_name = "PyBank"`** is written directly inside the class, not inside a method. That makes it a **class attribute**: it belongs to the class itself and is shared by every account. That's appropriate for a constant like the bank's name.
- **`__init__(self, owner, balance=0)`** receives the arguments given in `Account("ana", 100)`. `balance=0` is a default, so `Account("ben")` also works.
- **`self.owner = owner`** and **`self.balance = balance`** store the arguments on the new object.
- **`self.history = []`** gives *each account its own* empty list. Section 2 explains why it has to be created here and not somewhere else.
- **`deposit`** follows a shape you'll see over and over in exam code: first *validate* the input (refuse a negative amount by raising an error, see section 7), then *change the state*, then *return* something. Bugs often sit

in one of those three steps: a wrong comparison in the validation, the wrong attribute being changed, or the wrong thing returned.

- `__repr__` controls how the object is displayed in the REPL and in test failure messages. Without it you'd see something useless like `<__main__.Account object at 0x7f3a2c>`. With it, a failed test shows `Account('ana', 150)`, which tells you the state at a glance.

1.4 Forgetting `self.`

Missing self

Forgetting `self.` is a classic bug: `balance += amount` inside a method raises `UnboundLocalError`, and `history = []` creates a local variable that disappears.

Inside a method, a name *without* `self.` refers to an ordinary local variable of that method, which exists only while the method runs. So:

```
def deposit(self, amount):
    balance += amount            # BUG: which balance? There's no local variable called
        balance.
```

```
>>> acc.deposit(50)
UnboundLocalError: cannot access local variable 'balance' where it is not associated with a
    value
```

That one at least crashes loudly. This version is worse:

```
def reset_history(self):
    history = []                 # BUG: creates a local list, which is thrown away when the
        method ends
```

It runs without any error, but `self.history` never changes. If an object's state "doesn't stick", look for a missing `self.`.

2 The two big sharing traps

Both traps below are the same underlying problem: **one list (or dict) ends up shared by many objects**, when each object was supposed to have its own. They're among the most common planted bugs in class-based code.

2.1 Trap 1: a list stored as a class attribute

```
class Classroom:
    students = []                # BUG: one list for ALL classrooms

    def __init__(self, name):
        self.name = name

    def enrol(self, student):
        self.students.append(student)
```

```
>>> maths = Classroom("maths")
>>> art = Classroom("art")
>>> maths.enrol("Ana")
>>> art.students
['Ana']
```

We enrolled Ana in maths and she appeared in art. Here's why. When you read `self.students`, Python first looks for an attribute called `students` on the object itself. Nothing was ever assigned to `self.students`, so it doesn't find one there, and it falls back to the class attribute. There is only one class attribute, shared by every classroom. `append` modifies that single list. The fix is to create the list inside `__init__`, so each object gets its own:

```
def __init__(self, name):
    self.name = name
    self.students = []
```

Class attributes are fine for values you never modify in place, like numbers, strings and constants. The problem is specifically *mutable* values (lists, dicts, sets) that are changed with methods like `append`.

2.2 Trap 2: a mutable default argument

This one surprises even experienced programmers.

```
class Order:
    def __init__(self, lines=[]):      # BUG
        self.lines = lines
```

```
>>> o1 = Order()
>>> o1.lines.append("2 x coffee")
>>> o2 = Order()
>>> o2.lines
['2 x coffee']
```

A brand-new order already contains two coffees. Why? **Default values are created once, when Python first reads the `def` line, not each time the function is called.** So every call that doesn't pass `lines` receives *the same* list object. `o1` and `o2` share it. The standard fix is to use `None` as the default and create a fresh list inside the function:

```
def __init__(self, lines=None):
    self.lines = list(lines) if lines is not None else []
```

Read the second line as: "if the caller gave us some lines, make a copy of them as a list; otherwise, start with a new empty list". Whenever you see `=[]` or `={}` in a function's parameters, check it.

2.3 The quieter cousin: aliasing the caller's list

Aliasing the input

Also note `self.lines = lines` *aliases* the caller's list. If the spec says the object must not modify the input, copy it: `list(lines)`.

```
>>> todo = ["milk"]
>>> o = Order(todo)                 # with the version that does self.lines = lines
>>> o.lines.append("bread")
>>> todo
['milk', 'bread']
```

The caller's own list changed behind their back, because `self.lines` and `todo` are two names for the same list (Day 1, "aliasing"). `list(lines)` makes an independent copy. As a bonus, it also accepts tuples and other sequences.

3 Properties: attributes that run code

3.1 The problem they solve

Sometimes you want an attribute that *looks* like a normal value from the outside, but you want some code to run when it's read or changed. Two typical reasons:

- **validation:** refuse invalid values, for example a percentage above 100;
- **computed values:** a value calculated from other data whenever it's read, so it can never be out of date.

A **property** does this. To the user of the class, `p.value` and `p.value = 20` look like plain attribute access, but behind the scenes they call methods.

```
class Percentage:
    def __init__(self, value):
        self.value = value               # goes through the setter below

    @property
    def value(self):
        return self._value               # underscore: the real storage

    @value.setter
    def value(self, new):
        if not 0 <= new <= 100:
            raise ValueError("a percentage must be between 0 and 100")
        self._value = new                # writing self.value here = infinite recursion

    @property
    def fraction(self):                  # computed, read-only
        return self._value / 100
```

```
>>> p = Percentage(20)
>>> p.value
20
>>> p.fraction
0.2
>>> p.value = 75
>>> p.fraction             # automatically up to date
0.75
>>> p.value = 120
ValueError: a percentage must be between 0 and 100
```

How the pieces fit together:

- `@property` is a **decorator**: a line starting with `@` placed above a `def`, which changes how that function behaves. Here it turns the method `value` into something that runs when you *read* `p.value`.

- `@value.setter` marks the method that runs when you *assign* `p.value = something`.
- The real number is stored under a different name, `_value`. The leading underscore is a convention meaning "internal, don't touch from outside the class".
- `fraction` has no setter, so it's read-only: `p.fraction = 0.5` raises an error.

Two details are worth checking in exam code:

- In `__init__`, `self.value = value` goes *through the setter*, so the validation also applies when the object is created. If it were written `self._value = value`, invalid values would slip in at creation time.
- Inside the setter, the number must be stored in `self._value`. If the setter wrote `self.value = new`, that would call the setter again, which would call it again, and so on until Python gives up with a `RecursionError`.

4 `classmethod` and `staticmethod`

Normal methods receive the object as `self`. Two decorators change that.

4.1 `@classmethod`: alternative ways to create an object

A class method receives the **class** itself (by convention called `cls`) instead of an object. Its main use is providing extra ways of building an object, for example from a string or a saved file:

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    @classmethod
    def from_string(cls, s):     # alternative constructor; cls is the class
        x, y = s.split(",")
        return cls(int(x), int(y))
```

```
>>> p = Point.from_string("3,4")
>>> p.x, p.y
(3, 4)
```

`from_string` parses the text and then calls `cls(...)`, which is the same as calling `Point(...)`, so `__init__` runs as usual. You'll meet this pattern in the exams as `Tokenizer.load(text)` or `KVStore.deserialize(data)`: a method you call on the class to rebuild an object from a saved format.

4.2 `@staticmethod`: a plain function kept inside the class

A static method receives neither the object nor the class. It's just an ordinary function that lives in the class for tidiness, because it's logically related:

```
class Point:
    @staticmethod
    def distance(a, b):          # no self/cls: just a function in the class namespace
        return ((a.x - b.x) ** 2 + (a.y - b.y) ** 2) ** 0.5
```

You call it as `Point.distance(p, q)`.

5 Special (“dunder”) methods

5.1 What they are

You’ve already met two methods with funny names: `__init__` and `__repr__`. Methods whose names start and end with two underscores are called **dunder methods** (“double underscore”), or *magic methods*. You never call them directly. Python calls them for you when you use certain built-in syntax on your object. For example, if your class defines `__len__`, then `len(obj)` works on it. If it defines `__eq__`, then `obj1 == obj2` uses your rule for equality. This is how a custom class, like a cache or a linked list, can behave like a built-in list or dict. This matters in the exam because tests often use that built-in syntax: `len(cache)`, `key in store`, `list(tokens)`, `a == b`. If the dunder method behind the syntax has a bug, those tests fail, even though the line in the test looks completely innocent.

5.2 The important ones

Method	Called by	Notes
`__init__(self, ...)`	`Cls(...)`	returns None
`__repr__(self)`	`repr(x)`, REPL, inside lists	unambiguous; tests often check its exact format
`__str__(self)`	`str(x)`, `print(x)`, f-string	falls back to `__repr__`
`__eq__(self, other)`	`==`	return `NotImplemented` for foreign types
`__hash__(self)`	`hash()`, set/dict keys	**defining `__eq__` sets `__hash__` to None!**
`__lt__(self, other)`	`<`, `sorted()`	`sorted` only needs `__lt__`
`__len__(self)`	`len(x)`, truthiness	must return int ≥ 0
`__bool__(self)`	`if x:`	
`__iter__(self)`	`for v in x`, `list(x)`	often written as a generator with `yield`
`__getitem__(self, i)`	`x[i]`	raise `IndexError` / `KeyError`
`__setitem__`, `__delitem__`	`x[i] = v`, `del x[i]`	
`__contains__(self, v)`	`v in x`	
`__add__`, `__sub__`, `__mul__`	`+ - *`	`__rmul__` for `3 * v`

The rows that need some explanation:

- **`__repr__` vs `__str__`.** `__repr__` is the developer-facing description (ideally it looks like the code that would recreate the object). `__str__` is the user-facing one, used by `print`. If a class only defines `__repr__`, Python uses it for both.
- **`__eq__` and `NotImplemented`.** By default, `a == b` on two objects is only true if they’re the *very same* object. Defining `__eq__` lets you say “two points are equal if their coordinates are equal”. If `other` is something unrelated (say, a string), the convention is to return the special value `NotImplemented`, which tells Python “I don’t know how to compare with that”. Python then tries other options and finally answers `False`.
- **Why defining `__eq__` breaks hashing.** Sets and dict keys rely on a rule: *two objects that are equal must have the same hash* (section 5 of Day 3). Python’s default hash is based on the object’s identity. Once you’ve said that two *different* objects can be equal, that default would break the rule. So Python removes it, and your objects can no longer go into a set:

```
>>> {Money(5, "EUR")}        # a Money class with __eq__ but no __hash__
TypeError: unhashable type: 'Money'
```

The fix is to also define `__hash__`, built from *the same fields* that `__eq__` compares (see the example below).

- **`__len__` and truthiness.** If a class doesn't define `__bool__`, `if obj:` falls back to `len(obj) != 0`. So an empty cache object is *falsy*. Code that writes `if cache:` meaning "if the cache exists" will be surprised when the cache simply happens to be empty. That's the same "zero vs nothing" trap from Day 1.
- **`__contains__`** is what `in` calls. If it's missing, Python falls back to looping through `__iter__`. A `__contains__` that has *side effects* is a planted bug you'll meet in drill 6. For example, it might call a `get` method that also updates internal bookkeeping, so merely asking "is this in here?" changes the object.
- **`__rmul__`.** `v * 3` calls `v.__mul__(3)`. But `3 * v` first asks the integer 3 to multiply itself by a `v`, which it doesn't know how to do, so Python then calls `v.__rmul__(3)`, the "reflected" version.

5.3 A worked example

Here's a small class that uses several of these together:

```
class Vector:
    def __init__(self, x, y): self.x, self.y = x, y
    def __eq__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)
    def __hash__(self):
        return hash((self.x, self.y))
    def __iter__(self):
        yield self.x
        yield self.y
```

```
>>> Vector(1, 2) == Vector(1, 2)
True
>>> len({Vector(1, 2), Vector(1, 2)})     # equal vectors count once in a set
1
>>> x, y = Vector(3, 4)                   # unpacking works because of __iter__
>>> list(Vector(3, 4))
[3, 4]
```

- `isinstance(other, Vector)` checks whether `other` is a `Vector` (or a subclass of it, see section 6).
- `__eq__` compares the coordinates by packing them into tuples, a neat way of comparing several fields at once.
- `__hash__` hashes *the same tuple*, so equal vectors are guaranteed to have equal hashes.
- `__iter__` uses `yield`, which is explained in section 8.

6 Inheritance

Inheritance lets you define a new class based on an existing one. The new class (the *subclass*) automatically gets all the methods of the original (the *base class*), and can add new ones or replace some with its own version. Replacing a method is called **overriding** it.

```python
class Animal:
    def __init__(self, name):
        self.name = name
    def sound(self):
        raise NotImplementedError

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)        # forgetting this => no self.name => AttributeError
        self.breed = breed
    def sound(self):
        return "woof"
```

```
>>> rex = Dog("Rex", "beagle")
>>> rex.sound()
'woof'
>>> rex.name
'Rex'
>>> isinstance(rex, Animal)
True
```

- `class Dog(Animal)` declares that a `Dog` is a kind of `Animal`. That's why `isinstance(rex, Animal)` is `True`.
- `Animal.sound` raises `NotImplementedError`. That's a conventional way of saying "every kind of animal must provide its own `sound`".
- `Dog` defines its own `__init__`, which *replaces* `Animal.__init__`. The base class's setup (storing `self.name`) would therefore never run on its own. `super().__init__(name)` calls it explicitly. If that line were missing, `rex.name` would fail with `AttributeError: 'Dog' object has no attribute 'name'`.

7 Exceptions: signalling that something went wrong

7.1 What an exception is

An **exception** is Python's way of saying "something went wrong, and I can't carry on normally". When one is **raised**, the current function stops immediately. The exception travels back up through whichever functions called it, until some code **catches** it. If nothing catches it, the program stops and prints a traceback. You've already seen several built-in exceptions: `KeyError`, `IndexError`, `TypeError`, `ValueError`. You can also raise them yourself, with a message explaining what went wrong:

```python
def withdraw(self, amount):
    if amount > self.balance:
        raise ValueError(f"cannot withdraw {amount!r}: balance is {self.balance}")
    self.balance -= amount
```

Specs often say exactly which exception must be raised and when: "raises `ValueError` on malformed input", "raises `KeyError` if the key doesn't exist". Tests check this with `assertRaises` (Day 1, section 7), so raising the wrong type, or not raising at all, fails the test.

7.2 Catching exceptions: `try` / `except`

```
class InsufficientFunds(Exception):     # custom exception = subclass of Exception
    pass

try:
    risky()
except (KeyError, IndexError) as e:     # catch specific exceptions
    handle(e)
except ValueError:
    raise                               # re-raise the same exception
else:
    runs_if_no_exception()
finally:
    always_runs()

raise ValueError(f"bad value: {x!r}")
```

Piece by piece:

- **Custom exceptions.** `class InsufficientFunds(Exception): pass` creates your own kind of error. It inherits from `Exception`, and that's all it needs (`pass` means "nothing else to add"). It lets callers catch your specific error without catching unrelated ones.
- **`try:`** contains the code that might fail.
- **`except (KeyError, IndexError) as e:`** runs if one of the listed kinds of error happens inside the `try`. `e` is the exception object, which holds the message. Python checks the `except` clauses from top to bottom and runs the first one that matches.
- **`raise`** on its own, inside an `except`, re-raises the same exception. You use it when you want to react (for example undo something) but still let the error continue upward.
- **`else:`** runs only if the `try` block raised nothing.
- **`finally:`** always runs, error or not. It's used for cleanup.

Here's a concrete example of catching:

```
>>> def safe_int(text):
...     try:
...         return int(text)
...     except ValueError:
...         return None
...
>>> safe_int("42")
42
>>> safe_int("forty-two")
>>>
```

7.3 What `assertRaises` really checks

- Tests use `with self.assertRaises(ValueError):` — the exact **type** matters; a subclass also matches (`InsufficientFunds(ValueError)` would satisfy `assertRaises(ValueError)`).
- A bare `except:` or `except Exception:` that swallows errors is a common planted bug: the test expects an exception but the code returns `None` instead.
- `KeyError` is a subclass of `LookupError`, as is `IndexError`.

Exceptions form a family tree. `KeyError` and `IndexError` are both kinds of `LookupError`, and nearly everything is a kind of `Exception`. An `except` clause catches the type you name *and all its descendants*. That's why `except Exception:` is dangerous: it catches almost everything, including the errors a spec *requires* to escape. Here's how that turns into a bug:

```python
def parse_age(text):
    """Return the age as an int. Raise ValueError if text is not a number."""
    try:
        return int(text)
    except Exception:
        return None          # BUG: the spec wants the ValueError to reach the caller
```

The test `with self.assertRaises(ValueError): parse_age("abc")` fails, because the function quietly returns `None` instead.

8 Generators: producing values one at a time

8.1 What `yield` does

A function that contains the word `yield` is a **generator function**. It works differently from a normal function. Calling it doesn't run its body. Instead, it gives you a *generator*: an object that produces values lazily, one at a time, whenever something asks for the next one (a `for` loop, `list()`, ...). Each `yield` hands out one value and *pauses* the function, keeping all its local variables. When the next value is requested, it resumes right after the `yield`.

```python
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

```
>>> for x in countdown(3):
...     print(x)
...
3
2
1
>>> list(countdown(3))
[3, 2, 1]
```

8.2 Why it's useful for classes

This is the easiest way to write `__iter__` for your own container, which is what makes `for item in my_object` work. You don't need to build a list or write a separate class. You just walk through your internal data and `yield` each item. For a linked list (see the exercises), that's:

```python
def __iter__(self):
    node = self.head
    while node is not None:
        yield node.value
        node = node.next
```

8.3 One pass only

One-shot generators

A generator is exhausted after one pass: `g = countdown(3); list(g); list(g)` → `[3,2,1]`, `[]`.

```
>>> g = countdown(3)
>>> list(g)
[3, 2, 1]
>>> list(g)
[]
```

Once a generator has given out all its values, it's finished for good. A bug can hide here: if code stores a generator in a variable and loops over it twice (say, once to count the items and once to process them), the second loop silently sees nothing. When you need the values twice, turn them into a list first. (`__iter__` doesn't have this problem: each new `for` loop calls `__iter__` again and gets a fresh generator.)

9 Copying objects

The shallow vs. deep distinction from Day 3 applies to your own objects too:

```
import copy
b = copy.copy(a)        # shallow: new outer object, same inner objects
b = copy.deepcopy(a)    # recursive
```

`copy.copy(acc)` creates a new `Account` object, but its `history` attribute points to *the same list* as the original, so appending to one appends to both. `copy.deepcopy(acc)` copies the list too. You'll need this distinction for features like "take a snapshot" or "start a transaction", where a saved copy must stay unchanged while the live object keeps changing.

10 The toolbox: techniques the exercises rely on

10.1 `is` versus `==`: the same object, or equal values?

Python has two ways of asking "are these the same?", and they mean different things:

- `a == b` asks: **do they have equal values?** (For your own classes, that's whatever `__eq__` says.)
- `a is b` asks: **are they the very same object?** Two name tags on one thing (Day 1, aliasing).

```
>>> a = [1, 2]
>>> b = [1, 2]
>>> a == b          # same contents
True
>>> a is b          # but two separate lists
False
>>> c = a
>>> c is a          # c is just another name for a
True
```

Use `is` in two situations:

- **Comparing with `None`**: always write `x is None` / `x is not None`, never `x == None`. There's only one `None` object, so `is` is the precise question, and it can't be fooled by a class with a strange `__eq__`.
- **"Is this literally the same object?"**: for example, a transfer from an account to *itself* should be rejected. `if other is self:` catches exactly that case. `other == self` would also be true for two different accounts that happen to compare equal.

A trap related to section 2: `if items:` and `if items is not None:` are different. The first is false for an empty list, the second only for `None`. When a parameter's default is `None` and the caller may pass an empty list, you want the second:

```
def __init__(self, items=None):
    self.items = list(items) if items is not None else []
```

Read it as: "if the caller gave us something, make our own copy of it with `list(...)`; otherwise start with a fresh empty list". The copy matters for the aliasing reason from section 2: we don't want to share the caller's list.

10.2 `any` and `all`

`any(...)` is `True` if **at least one** of the values is true; `all(...)` is `True` if **every** value is true. They're almost always used with a generator expression (Day 1, section 8):

```
>>> nums = [3, 8, 1]
>>> any(n > 5 for n in nums)         # is there a number bigger than 5?
True
>>> all(n > 0 for n in nums)         # are they all positive?
True
>>> any(x == 7 for x in [])          # nothing to check: no
False
>>> all(x == 7 for x in [])          # nothing contradicts it: yes!
True
```

Both stop as soon as they know the answer: `any` at the first true value, `all` at the first false one. "Is `value` in my linked list?" is `any(v == value for v in self)`, which works as soon as the class has an `__iter__`.

10.3 `iter()`, and two ways to write `__iter__`

`__iter__` must return an **iterator**: an object that hands out values one at a time to a `for` loop. There are two common ways to write it. **1. As a generator, with `yield`** (section 8). Calling a method that contains `yield` automatically produces an iterator:

```
def __iter__(self):
    yield self.x
    yield self.y
```

2. By handing over the iterator of a list you already have, with the built-in `iter()`:

```
def __iter__(self):
    return iter(self.songs)     # "loop over me = loop over my songs list"
```

`iter(some_list)` returns an iterator over that list. Don't mix the two styles: `yield self.songs` would hand out the *whole list* as one single value, so `list(playlist)` would be `[['a', 'b']]` instead of `['a', 'b']`.

10.4 Vector length: `**` and `__abs__`

`**` is "to the power of": `3 ** 2` is 9, and `x ** 0.5` is the square root of `x` (the power ½). The length of a vector `(x, y)` is, by Pythagoras, `(x ** 2 + y ** 2) ** 0.5`; for `(3, 4)` that's `(9 + 16) ** 0.5 = 5.0`. `abs(obj)` calls `obj.__abs__()`, the dunder for "absolute value", which for a vector naturally means its length. The same pattern as `len(obj)` → `__len__`.

10.5 Updating two variables at once: `prev, cur = cur, cur.next`

Unpacking (Day 1, section 8) has a property that's essential for linked lists: **the whole right-hand side is computed first, then assigned**. So

```
prev, cur = cur, cur.next
```

means "the new `prev` is the old `cur`, and the new `cur` is the old `cur.next`", both based on the values *before* the line ran. Written as two lines, `prev = cur` then `cur = cur.next`, it happens to work too. But the order of the lines would matter, and swapping them breaks it. The one-line form can't be got wrong that way.

10.6 Working with a linked list

A linked list (what it is: see the ex03 background under "Exercises" below) keeps a reference to its first node, `head`, and usually also its last node, `tail`, so adding at the end doesn't require walking the whole chain. Most operations are short, but each has one or two edge cases, nearly always "the list is empty" or "the list has exactly one node". Draw the boxes and arrows on paper and check those cases for every method. **Walking the list.** Start at the head and follow `next` until you fall off the end (`None`):

```
cur = self.head
while cur is not None:
    ... cur.value ...
    cur = cur.next
```

Adding at the end (with a tail reference): link the new node after the current tail, then move `tail` to it. If the list was empty, the new node is both the head and the tail. **Adding at the front:** the new node's `next` is the old head, and the new node becomes the head. `Node(value, self.head)` does the first half, because `Node`'s second parameter is `next`. If the list was empty, the new node is also the tail. **Removing a node** means making the node *before* it skip over it: `prev.next = cur.next`. That's why removal walks with two variables, `prev` and `cur`, one step apart. If the node to remove is the head, there is no `prev`, and you move `self.head` instead. If it's the tail, `tail` must move back to `prev`. **Reversing** turns every arrow around. You walk along the list with `prev` and `cur`, and at each node you make it point back to `prev`. But once you change `cur.next` you've lost the way forward, so you must save it first:

```
start:   prev=None   cur=A       A -> B -> C -> None
save nxt = B,  set A.next = None (prev),  move: prev=A, cur=B
         None <- A    B -> C -> None
save nxt = C,  set B.next = A,  move: prev=B, cur=C
         None <- A <- B    C -> None
save nxt = None,  set C.next = B,  move: prev=C, cur=None   (stop)
         None <- A <- B <- C        the new head is prev = C
```

Three steps per node, always in this order: **save next, turn the arrow, move both pointers forward**. Don't forget that the old head becomes the new tail. **Negative indices.** To support `ll[-1]` like a normal list, turn a negative index into the equivalent positive one by adding the length: in a list of 5, `-1` becomes 4 and `-5` becomes `0`. Then check the range with a chained comparison (Day 1): `if not 0 <= index < len(self): raise IndexError(...)`.

10.7 Methods that return `self`

A method can end with `return self`, giving back the object it was called on. That allows **chaining** calls:

```
playlist.add("a").add("b").add("c")    # each add returns the playlist, so the next add is
    called on it
```

Exercises

Practice lab

Work in `exercises/`. Read each docstring as the spec. Run `python run.py 4` after every change; peek at `solutions/` only after 15 minutes stuck.

File	Type	What
`exercises/ex01_bank.py`	implement	BankAccount with custom exception, history, transfer
`exercises/ex02_vector.py`	implement	Vector2D with operators, hashing, iteration
`exercises/ex03_linked_list.py`	implement	LinkedList with `__len__`/`__iter__`/`__getitem__`/`__contains__`, reverse
`exercises/ex04_fix_me.py`	**debug**	6 classes, one bug each (the classic object-model traps)

Background for each:

- **ex01** is a bank account with deposits, withdrawals and transfers between two accounts. Think about the transfer: if it fails halfway (not enough money), *neither* account may have changed. So check everything first, and only then modify anything.
- **ex02** is a 2-D vector, a pair of numbers `(x, y)` that you can add, subtract and scale like arrows on a map. It practises operator dunders and the `__eq__`/`__hash__` pair from section 5.
- **ex03** is a **linked list**, a different way to store a sequence. Instead of one block of positions like a Python list, it's a chain of small `Node` objects. Each node holds one value and a reference to the next node (`node.next`), and the list itself only remembers the first node (the **head**). The last node's `next` is `None`. There are no positions, so to reach the 5th item you start at the head and follow `next` four times. **Reversing** the list means walking along it and turning every `next` arrow around, a classic exercise in updating references in the right order. This is the warm-up for the cache in drill 6, which uses a list where each node points both forward and backward.
- **ex04** has six small classes, each containing one of the traps from this document.

Which tools each part needs (section numbers in brackets):

Exercise	Tools
ex01 `BankAccount`	a custom exception class, `raise` (7), `other is self` (10), `history.append((op, amount))`, `__repr__` with `!r` (5)
ex02 `Vector2D`	`isinstance` and `NotImplemented` (5), `__rmul__` (5), `**` and `__abs__` (10), `__hash__` from a tuple (5), `__iter__` with `yield` (8)
ex03 `LinkedList`	walking, adding and removing nodes, reversing (10), `prev, cur = cur, cur.next` (10), negative indices (10), any (10), `__len__`/`__iter__`/`__contains__`/`__getitem__` (5)
ex04 fix-me	class attributes and mutable defaults (2), `x is not None` (10), `__hash__` (5), `super().__init__` (6), property setters (3), `iter()` (10)

```
python run.py 4
```

End-of-day checklist

Tick every box

- [ ] I can spot a mutable class attribute or mutable default argument instantly.
- [ ] I know defining `__eq__` makes instances unhashable unless I add `__hash__`.
- [ ] I can write `__iter__` with `yield`.
- [ ] I know `super().__init__(...)` must be called in a subclass that overrides `__init__`.

Exam lens

- The broken implementation is often a class: inspect `__init__`, shared state, and dunders first.
- A missing `super().__init__` or a property setter that assigns to itself is a quick win.
- Match the exact exception type the tests expect; a subclass also satisfies `assertRaises`.

Tonight's bug journal

Where	Symptom	Root cause / fix

One row per bug. Re-read this page on Day 10.